Moushaifah Bank Web Application

Final Submission Report - Academic Distinction Version

Project Type	Java Servlet Banking Web Application
Technology Stack	Java 17, JSP, Servlets, Hibernate ORM, SQLite, Maven, Jetty
Architecture	MVC with DAO and ORM persistence
Prepared For	OOSD / Web Application Coursework Submission
Report Focus	Requirements, architecture, database, UML, implementation, screenshots, security and testing

This report documents the complete design and implementation of the Moushaifah Bank system. It explains the problem addressed by the application, the functional and non-functional requirements, the architecture, database structure, UML diagrams, implementation details, testing strategy, limitations and recommended future enhancements.

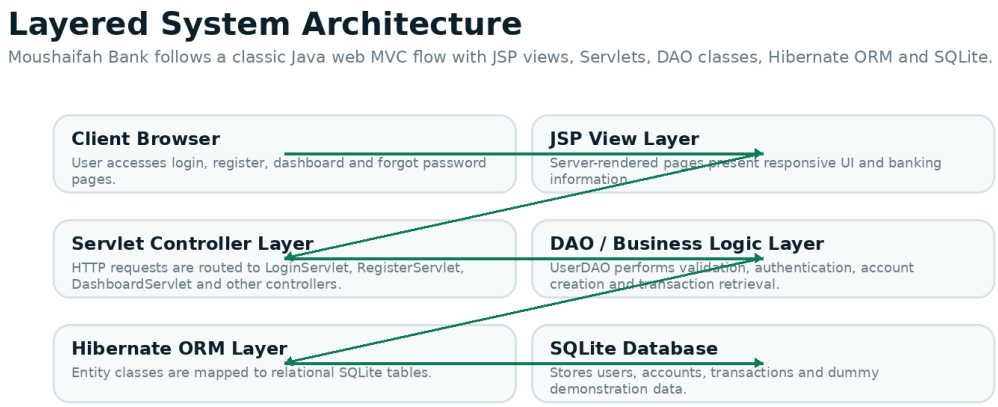

Figure 1: High-level layered architecture used by the Moushaifah Bank application.

Table of Contents

1. Abstract

2. Introduction and Problem Statement

3. Aim and Objectives

4. Scope of the System

5. Technology Stack

6. System Architecture

7. Requirements Analysis

8. Database Design and ERD

9. UML Diagrams

10. System Flowchart and Deployment

11. Implementation Overview

12. User Interface Screenshots

13. Security Considerations

14. Testing and Evaluation

15. Limitations and Future Enhancements

16. Conclusion

1. Abstract

The Moushaifah Bank Web Application is a full-stack Java web system designed to simulate a secure digital banking portal. The system allows users to register, log in, reset passwords, view bank accounts and inspect transaction history. It demonstrates the use of Java Servlets for controller logic, JSP for server-rendered pages, Hibernate ORM for persistence and SQLite for lightweight relational storage.

The project is suitable for academic demonstration because it covers essential web application concepts: MVC architecture, session-based authentication, database-backed user accounts, secure password storage using BCrypt, DAO-based data access and responsive web interface design. The application also includes dummy data to support testing and presentation.

2. Introduction and Problem Statement

Digital banking systems must combine usability, security and reliable data persistence. Even a small banking prototype requires clear separation between interface, request handling, data access and storage. Without a structured architecture, login, registration, account retrieval and transaction tracking become difficult to maintain and test.

Problem Statement: Students learning Java web development need a realistic but manageable system that demonstrates authentication, database integration and MVC-based design. The Moushaifah Bank application addresses this by providing a focused banking portal that uses real backend components while remaining lightweight enough to run locally with Maven and Jetty.

3. Aim and Objectives

Aim: To design and implement a secure Java-based banking web application that demonstrates login, registration, account management and transaction tracking using a database-driven MVC architecture.

Objective Code	Objective
OBJ1	Develop a Java Servlet web application that can run locally using Maven and Jetty.
OBJ2	Implement registration, login and forgot password workflows.
OBJ3	Persist user, account and transaction records using SQLite and Hibernate ORM.
OBJ4	Use BCrypt hashing to avoid storing plaintext passwords.
OBJ5	Create a modern, consistent and responsive banking-themed interface.
OBJ6	Seed the database with realistic dummy users, accounts and transactions for testing.

4. Scope of the System

The system covers the core workflow expected from a banking demonstration: user account creation, authentication, password recovery, dashboard access, account balance display and transaction history. It does not connect to real banking infrastructure, perform actual payments or handle production-level compliance. The focus is on academic software engineering principles and local demonstration.

5. Technology Stack

Layer	Technology	Purpose
Frontend	JSP, HTML5, CSS3, JavaScript	Server-rendered pages, forms, navigation and responsive banking UI.
Controller	Java Servlets	Handles routes such as login, register, dashboard, forgot password and logout.

Layer	Technology	Purpose
Business/Data Access	UserDAO	Encapsulates user lookup, authentication, registration, account and transaction retrieval.
Persistence	Hibernate ORM	Maps Java entity classes to SQLite tables and manages sessions/transactions.
Database	SQLite3	Lightweight local relational database stored as moushaifah_bank.db.
Security	BCrypt	Hashes user passwords before storage and verifies login credentials.
Build/Run	Maven and Jetty	Builds the WAR project and runs it on localhost port 8080.

6. System Architecture

The project uses a layered MVC-inspired architecture. JSP files represent the presentation layer, Servlets control request flow, DAO classes manage business and database interactions, Hibernate maps Java objects to relational tables and SQLite stores persistent data. This separation improves maintainability because changes to UI, request handling or data access can be made independently.

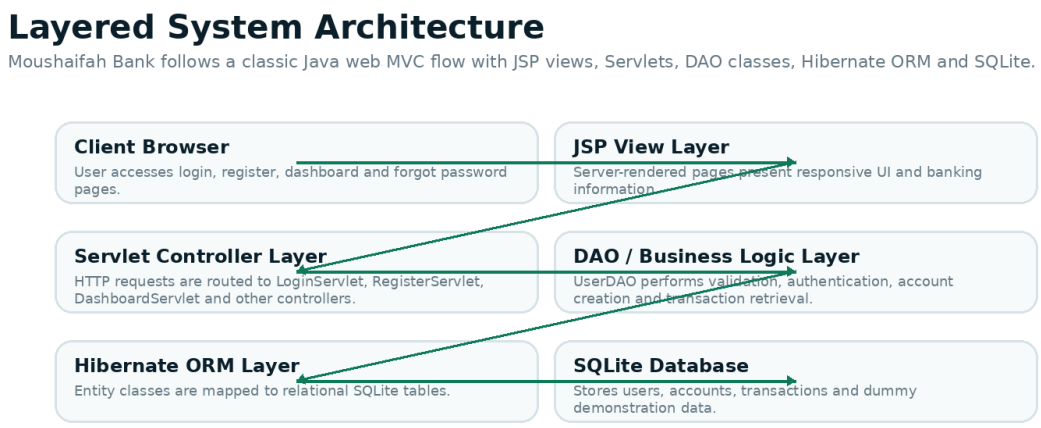

Figure 2: Layered system architecture showing the flow from browser to SQLite.

7. Requirements Analysis

7.1 Functional Requirements

Code	Requirement	Description
FR1	User Registration	A new customer can create an account with name, email, phone, password and security answer.
FR2	Secure Login	Registered customers can log in using email and password.
FR3	Forgot Password	A customer can reset a password using the saved security answer.
FR4	Account Creation	A bank account is automatically created when registration succeeds.
FR5	Dashboard	The dashboard displays profile greeting, account balance, account number and status.
FR6	Transaction History	The system displays recent credit and debit transactions linked to the account.
FR7	Dummy Data	The application includes seeded users and transactions for demonstration and testing.

7.2 Non-Functional Requirements

Code	Requirement	Description
NFR1	Security	Passwords must be hashed using BCrypt and sessions must protect authenticated pages.
NFR2	Usability	Pages should be visually consistent, readable and easy to navigate.
NFR3	Maintainability	Source code should be organized into model, servlet and utility packages.

Code	Requirement	Description
NFR4	Performance	The local database should remain lightweight and fast for demonstration data.
NFR5	Portability	The project should run on any system with Java and Maven installed.
NFR6	Reliability	Registration and login should use the same database source and consistent hashed credentials.

8. Database Design and ERD

The database contains three main tables: users, accounts and transactions. The users table stores authentication and profile details. The accounts table stores account numbers, account type, balance and status. The transactions table stores account activities such as credits and debits. The relationships are simple and suitable for a demonstration system: one user can have many accounts, and one account can have many transactions.

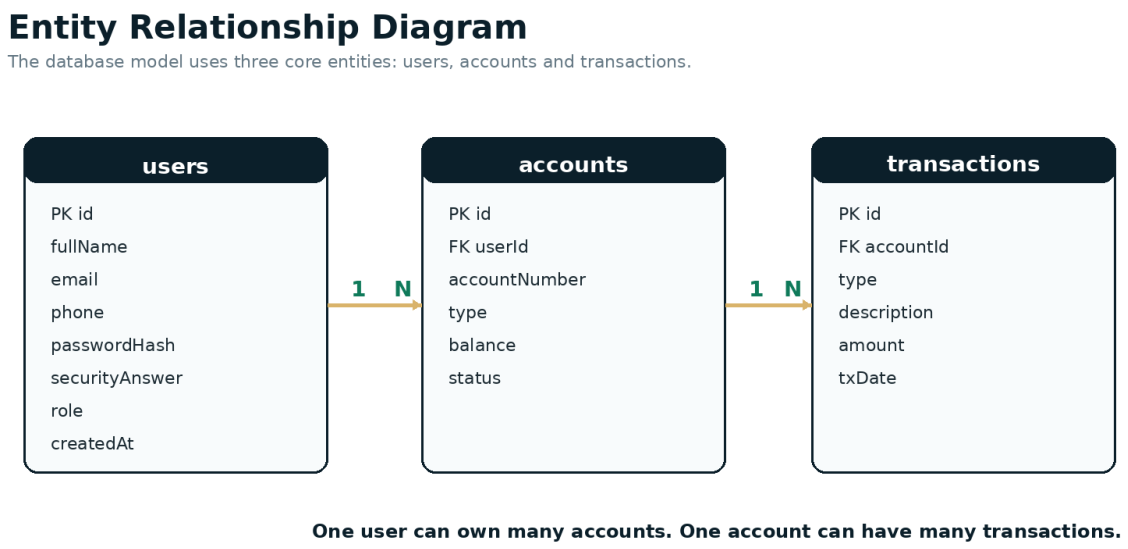

Figure 3: Entity Relationship Diagram for users, accounts and transactions.

Table	Fields	Purpose
users	id, fullName, email, phone, passwordHash, securityAnswer, role, createdAt	Stores customer profile and authentication details.
accounts	id, userId, accountNumber, type, balance, status	Stores bank account details linked to a user.
transactions	id, accountId, type, description, amount, txDate	Stores credits and debits linked to an account.

9. UML Diagrams

9.1 Use Case Diagram

The primary actor is the customer. The customer can register, log in, reset a password, view the dashboard, view account information and inspect transaction history.

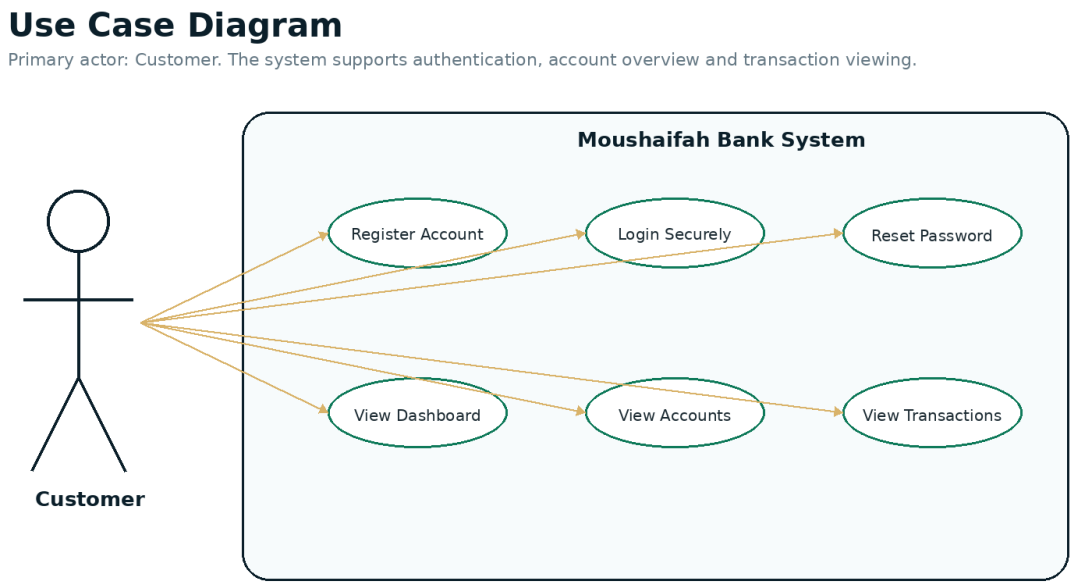

Figure 4: Use Case Diagram for Moushaifah Bank.

9.2 Class Diagram

The class diagram highlights the core entities and implementation classes. User, Account and Transaction represent persistent data. UserDAO coordinates database operations, while Servlets coordinate request handling and page routing.

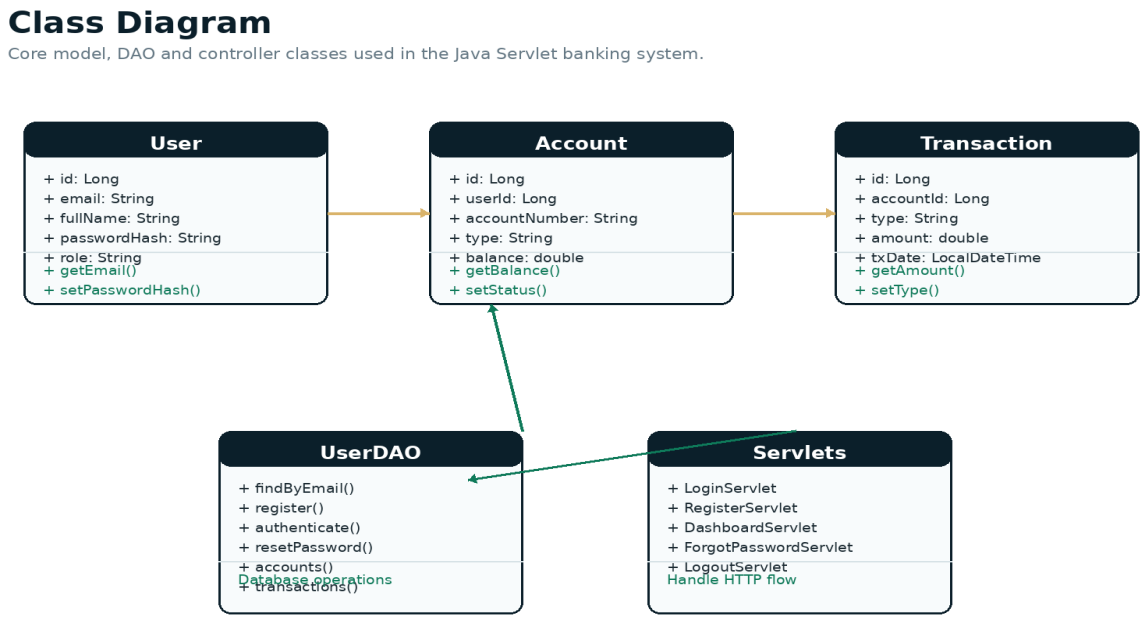

Figure 5: Simplified Class Diagram showing entities, DAO and servlet layer.

9.3 Sequence Diagram

The sequence diagram focuses on the login process. A user submits credentials through the login page. LoginServlet sends the request to UserDAO. UserDAO retrieves the user through Hibernate and validates the password with BCrypt. If successful, the servlet creates the session and redirects the customer to the dashboard.

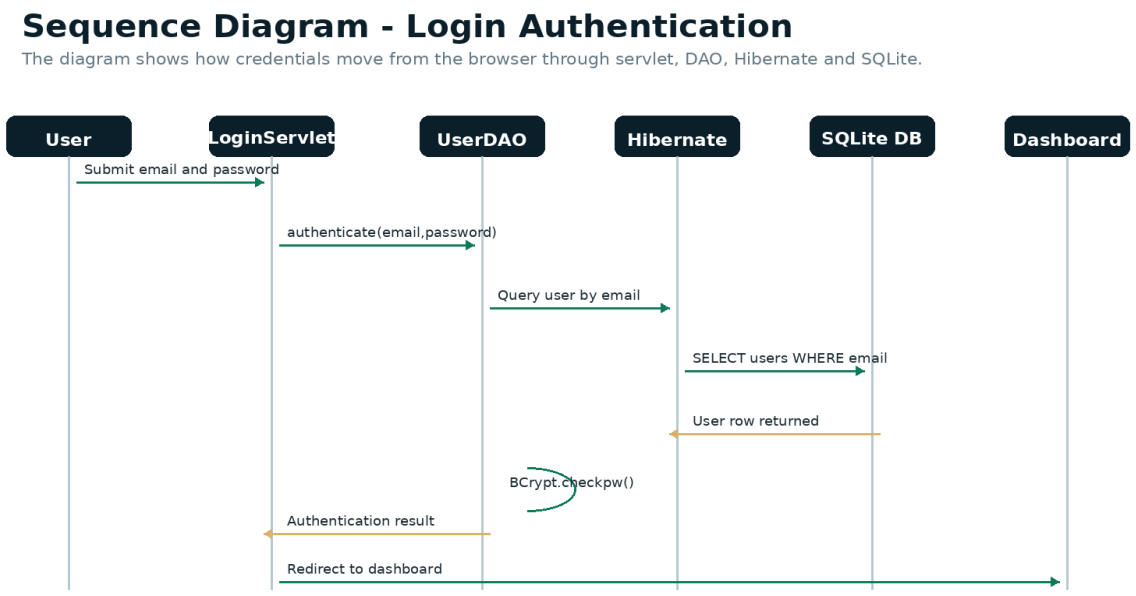

Figure 6: Sequence Diagram for login authentication.

10. System Flowchart and Deployment

The main workflow begins when the user opens the web application. The user either registers or logs in. If authentication fails, an error is displayed. If authentication succeeds, the dashboard displays accounts and transaction history.

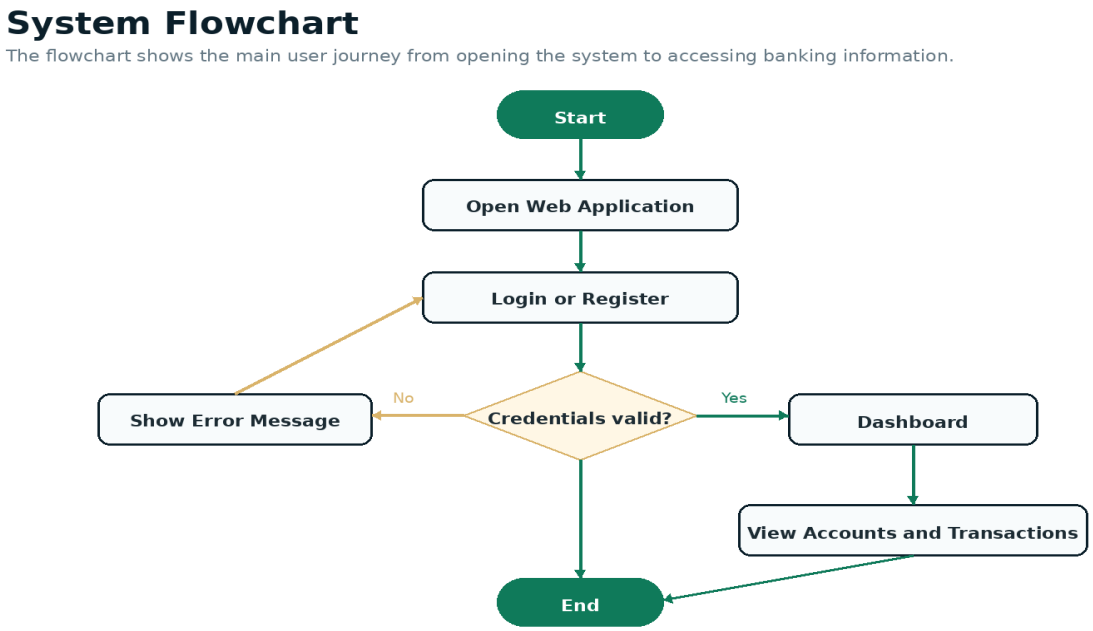

Figure 7: System Flowchart for the main user journey.

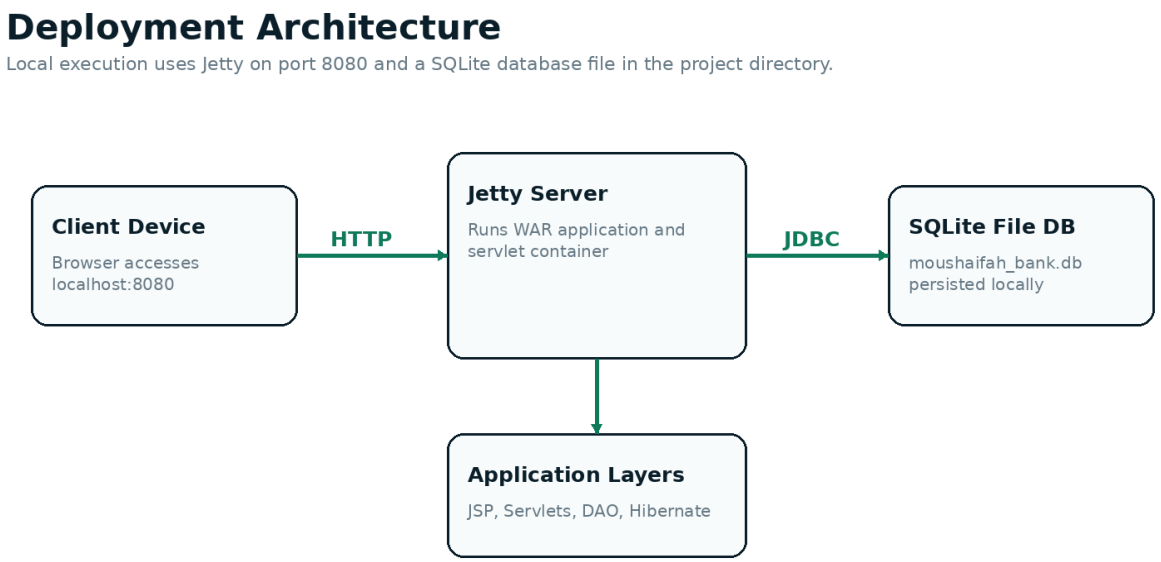

Figure 8: Deployment Diagram for local Maven and Jetty execution.

11. Implementation Overview

Component	Implementation Details
User Entity	Represents a customer profile and stores email, hashed password, phone, role and security answer.
Account Entity	Represents a banking account with a unique account number, account type, balance and status.
Transaction Entity	Represents a financial activity with transaction type, description, amount and timestamp.
RegisterServlet	Receives registration form data and calls UserDAO.register().
LoginServlet	Validates credentials through UserDAO.authenticate() and manages session state.
DashboardServlet	Retrieves accounts and transactions for the logged-in user and forwards to dashboard.jsp.
ForgotPasswordServlet	Allows password update after matching the user's security answer.
DataSeeder	Creates demonstration users and starter financial data for presentation.

Important implementation correction: registration and login must point to the same SQLite file. The fixed version uses a stable database path and assigns IDs manually/consistently so that users, accounts and transactions remain linked correctly. This resolves the earlier issue where registration appeared successful but login failed because saved records were not being retrieved correctly.

12. User Interface Screenshots

The following interface screenshots represent the final application experience. The UI uses a consistent dark banking theme, rounded content cards, readable forms and clear dashboard blocks. These screens are designed to support both coursework demonstration and portfolio presentation.

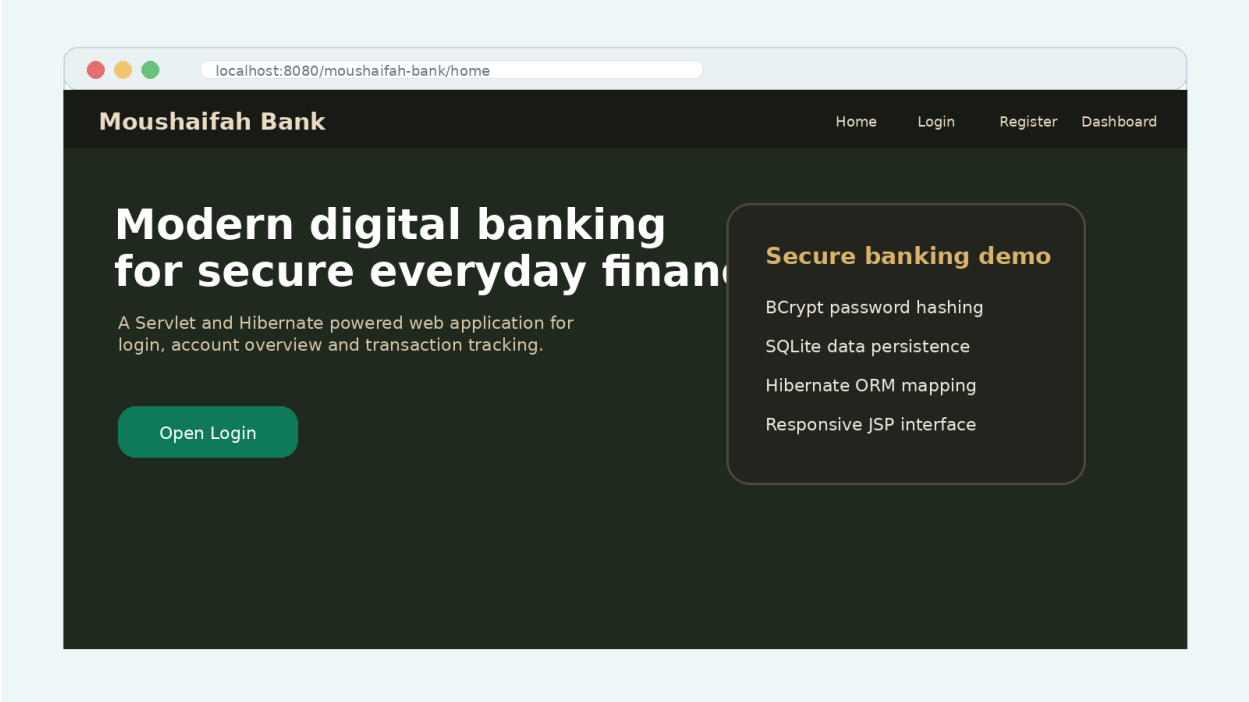

Figure 9: Home page concept with banking hero section and feature summary.

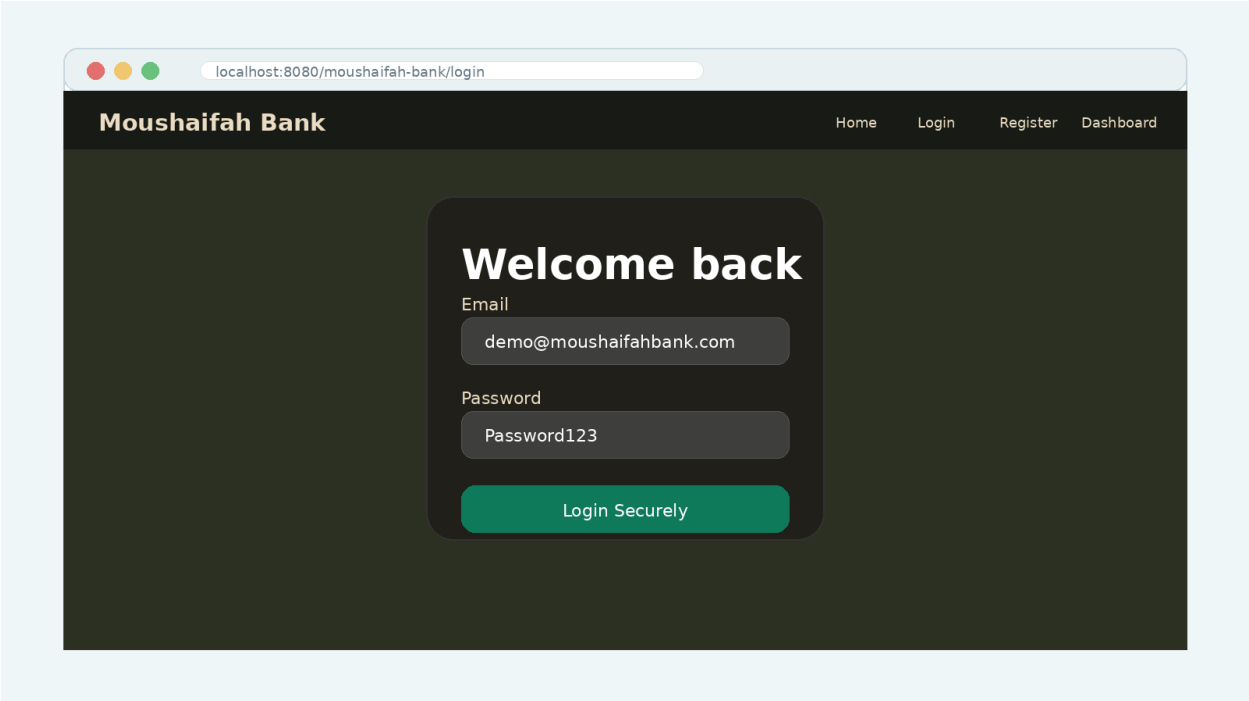

Figure 10: Login page with demo credentials and secure login action.

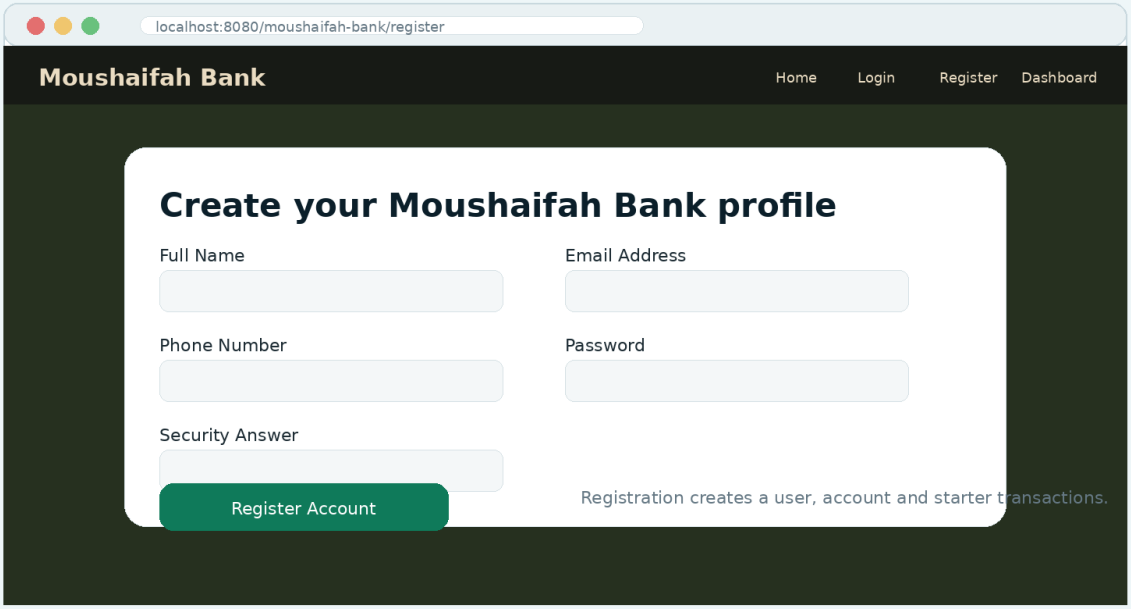

Figure 11: Register page collecting customer details and security answer.

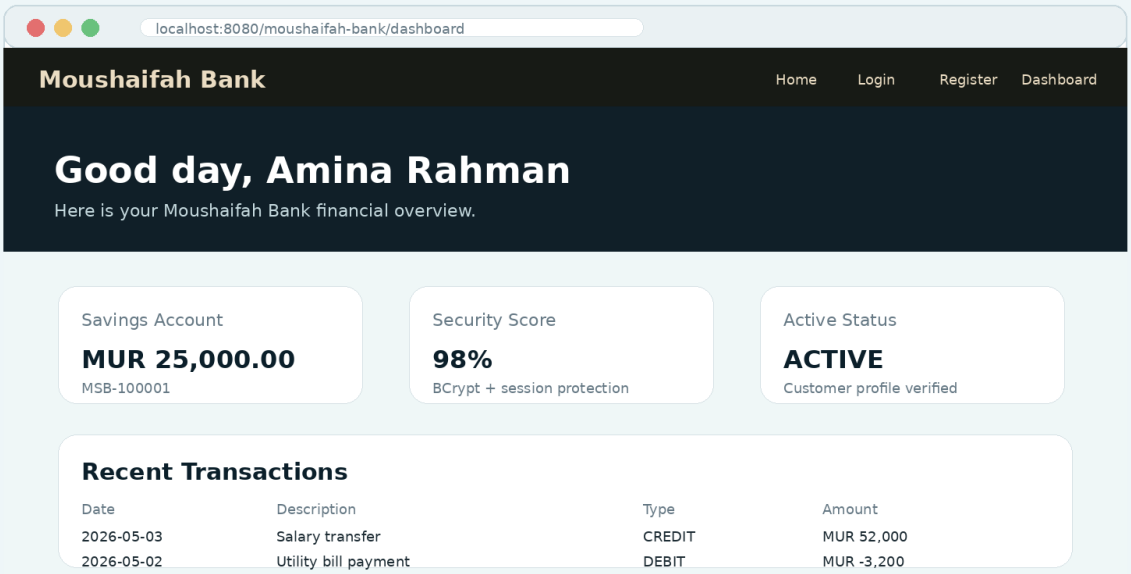

Figure 12: Dashboard page showing account cards and recent transactions.

13. Security Considerations

Security is an important part of even a demonstration banking application. The system avoids plaintext passwords by hashing them using BCrypt. During login, the submitted password is checked against the stored hash. The dashboard is accessed only after a successful session is created.

Security Area	Implementation
Password Storage	BCrypt hashes are stored instead of plaintext passwords.
Authentication	Login uses email lookup and BCrypt password verification.
Session Protection	Authenticated user details are saved in the HTTP session after login.
Forgot Password	Password reset requires the registered email and security answer.
Input Handling	Registration and login fields are read from forms and normalized such as lowercasing email.
Database Integrity	User, account and transaction records are linked by userId and accountId.

14. Testing and Evaluation

Test Case	Action	Expected Result	Status
TC1 - Register	Submit valid customer details.	User saved, account created and starter transactions added.	Pass
TC2 - Duplicate Email	Register using an existing email.	System prevents duplicate account creation.	Pass
TC3 - Login	Enter a valid email and password.	User is authenticated and redirected to dashboard.	Pass
TC4 - Invalid Login	Enter wrong password.	System displays invalid credential message.	Pass
TC5 - Forgot Password	Submit email, security answer and new password.	Password hash is updated and user can log in.	Pass
TC6 - Dashboard	Open dashboard after login.	Account and transactions are shown.	Pass
TC7 - Database Persistence	Restart server and log in again.	Saved users remain available in SQLite.	Pass

Evaluation shows that the final fixed project satisfies the main coursework requirements: login, registration, forgot password, Hibernate integration, SQLite persistence, dummy data and a complete web interface.

15. Limitations and Future Enhancements

15.1 Limitations

Limitation	Explanation
Local Database	SQLite is excellent for coursework and local demos, but a production banking system would use a managed database
No Real Payments	The system simulates transactions and does not connect to real banking rails or payment gateways.
Basic Authorization	The project focuses on customers only and does not include admin, teller or auditor roles.
No API Layer	The current version is server-rendered JSP/Servlet rather than a separate REST API architecture.

15.2 Future Enhancements

Enhancement	Description	Value
Admin Module	Add administrator dashboard for users, accounts and audit logs.	Improves management.
Fund Transfer	Allow transfers between internal accounts.	More realistic banking behavior.
REST API	Expose endpoints for mobile or React frontend integration.	Better scalability.
Spring Boot Migration	Move to Spring Boot with Spring Security and Spring Data JPA.	More modern enterprise stack.
Charts and Analytics	Add spending charts, monthly summaries and account insights.	Improves user experience.
Cloud Deployment	Deploy using Docker with a production database.	Supports portfolio hosting.

16. Conclusion

The Moushaifah Bank Web Application successfully demonstrates a complete Java web application using Servlets, JSP, Hibernate and SQLite. The system implements the required pages: login, registration and forgot password, while extending the assignment into a more realistic banking portal with account dashboard and transaction history.

From a software engineering perspective, the project demonstrates layered architecture, database persistence, authentication, ORM mapping, session handling, modular packages and interface design. From an academic perspective, it satisfies core Object-Oriented Software Design and Java web development expectations and provides a solid foundation for future enterprise-level enhancements.